Day 9 – Testing with pytest-django

Goals: Write a meaningful test suite covering the expiry flow and webhook dispatch.

```
uv add pytest-django pytest-mock
```

`pytest.ini` (project root):

```
[pytest]
DJANGO_SETTINGS_MODULE = gamekey_platform.settings
python_files = test_*.py
```

`games/tests/__init__.py` — empty file.

`games/tests/test_webhook.py`:

```
import pytest
from django.contrib.auth.models import User
from django.utils import timezone
from datetime import timedelta
from unittest.mock import patch, MagicMock
from django.core.management import call_command
from games.models import Publisher, Game, GameKey, WebhookDeliveryLog


@pytest.fixture
def publisher_user(db):
    user = User.objects.create_user(username='pub_user', password='pass
    publisher = Publisher.objects.create(
        name='Test Publisher',
        webhook_url='https://example.com/webhook',
        webhook_secret='supersecret',
        user=user,
    )
    return publisher


@pytest.fixture
def expired_game_key(db, publisher_user):
    game = Game.objects.create(
        title='Epic Game',
        publisher=publisher_user,
        price='29.99',
    )
    return GameKey.objects.create(
        key_string='TEST-KEY-0001',
        game=game,
        status='active',
        expires_at=timezone.now() - timedelta(hours=1),
        owner=publisher_user.user,
    )
```

```
@pytest.mark.django_db
@patch('games.tasks.send_expiry_webhook_async.delay')
def test_management_command_dispatches_tasks(mock_delay, expired_game_k
    call_command('check_expired_keys')
    assert mock_delay.called
    call_args = mock_delay.call_args
    assert call_args.kwargs['game_key_str'] == 'TEST-KEY-0001'


@pytest.mark.django_db
@patch('games.tasks.requests.post')
def test_webhook_task_logs_success(mock_post, publisher_user, expired_g
    from games.tasks import send_expiry_webhook_async
    mock_response = MagicMock()
    mock_response.status_code = 200
    mock_post.return_value = mock_response

    send_expiry_webhook_async(
        publisher_id=publisher_user.id,
        game_key_str='TEST-KEY-0001',
        game_title='Epic Game',
        expired_at_iso=expired_game_key.expires_at.isoformat(),
        attempt=0,
    )

    log = WebhookDeliveryLog.objects.get(game_key='TEST-KEY-0001')
    assert log.success is True
    assert log.response_status == 200


@pytest.mark.django_db
@patch('games.tasks.requests.post')
def test_webhook_task_logs_failure(mock_post, publisher_user, expired_g
    from games.tasks import send_expiry_webhook_async
    mock_post.side_effect = Exception('Connection refused')

    with pytest.raises(Exception):
        send_expiry_webhook_async.apply(kwargs=dict(
            publisher_id=publisher_user.id,
            game_key_str='TEST-KEY-0001',
            game_title='Epic Game',
            expired_at_iso=expired_game_key.expires_at.isoformat(),
            attempt=0,
        ))

    log = WebhookDeliveryLog.objects.get(game_key='TEST-KEY-0001')
    assert log.success is False
    assert 'Connection refused' in log.error_message

pytest --cov=games --cov-report=term-missing
```

🎓 Instructor Teaching Plan

⏱ Time Breakdown (90 min)

Segment	Duration	Activity
Testing philosophy	15 min	What to test and what not to test. Unit tests vs integration tests. The testing pyramid. Why we mock external I/O (network calls, time). What does "70% coverage" actually mean — and what it doesn't mean.
pytest-django setup	10 min	Install `pytest-django` and `pytest-mock`. Write `pytest.ini`. Explain `DJANGO_SETTINGS_MODULE`. Explain `@pytest.mark.django_db` — by default, tests can't access the DB.
Fixtures	15 min	Walk through `publisher_user` and `expired_game_key` fixtures. Explain: fixtures provide reusable setup, run fresh for each test. The `db` fixture enables DB access. Show how fixtures compose (one fixture depends on another).
Walk through the 3 tests	30 min	Test 1: management command dispatches tasks (mock `.delay`). Test 2: task logs success (mock `requests.post`). Test 3: task logs failure (mock raises exception). For each: read the assertion aloud, explain what it proves.
Run coverage + add one more	10 min	Run `pytest --cov=games --cov-report=term-missing`. Read the report together. Challenge students to add a test for the `create_order` view.
Q&A + checkpoint	10 min	

💡 Key Concepts to Introduce

- **Unit vs integration tests** — unit: test one function/method in isolation, mock everything external. Integration: test multiple components working together. Our tests are mostly unit tests with a real DB (pytest-django handles isolation).
- **`@pytest.fixture`** — a function that provides a value to tests. Runs before each test, tears down after. Fixtures can depend on other fixtures.
- **`@pytest.mark.django_db`** — grants DB access for that test. Each test gets a transaction that's rolled back at the end (no inter-test contamination).
- **`@patch(target)`** — replaces a name in the target module with a `MagicMock` for the duration of the test. The target must be the import path *where the name is used*, not where it's defined.
- **`MagicMock`** — an object that accepts any attribute access or method call and returns another mock. Set `.return_value` to control what it returns, `.side_effect` to make it raise an exception.
- **Coverage** — the percentage of code lines executed by at least one test. 70% is a minimum; it doesn't mean the logic is correct, just that the lines ran.
- **`task.apply(kwargs=dict(...))`** — runs the Celery task synchronously *without* retries. Useful for testing the task body directly without the Celery retry machinery.

⚠ Common Mistakes to Address

- **Wrong `@patch` target** — the most common testing mistake. `@patch('games.tasks.requests.post')` patches `requests.post` *as imported in `games/tasks.py`*. Patching `requests.post` globally won't work if `tasks.py` has already imported it. Rule: patch where it's used.

- **Not using `@pytest.mark.django_db`** — the test appears to pass (no assertion errors) because the fixture never hits the DB, but actually the DB wasn't accessible and the test proved nothing.
- **Calling `.delay()` in tests** — this actually queues the task to Redis (if a broker is running) or raises `OperationalError` (if not). Always use `@patch` or `CELERY_TASK_ALWAYS_EAGER = True` in tests.
- **Not asserting on the right thing** — `assert mock_delay.called` tells you the function was called, but `call_args.kwargs['game_key_str'] == 'TEST-KEY-0001'` verifies the right arguments were passed. Encourage students to assert on the *what*, not just the *whether*.
- **Shared state between tests** — pytest-django wraps each test in a transaction and rolls it back. Students should not rely on data from a previous test existing.

? Check for Understanding

> "Why do we `@patch('games.tasks.requests.post')` rather than `@patch('requests.post')`?"
>
> **Answer:** We patch where the module is imported and used, not where it is defined. In `games/tasks.py`, the code uses `requests.post`. If we patched `requests.post` globally, `games/tasks.py` would still use its local, unpatched reference to the real `requests.post` function.
>
> "What does `mock_post.side_effect = Exception('Connection refused')` do differently from `mock_post.return_value = ...`?"
>
> **Answer:** `.return_value` makes the mock function return a specific value (like a mock response object) when called. `.side_effect` raises the specified exception when the mock function is called, allowing us to test how the code handles connection failures or timeouts.
>
> "If a test has 100% line coverage, does that mean it's bug-free? Why or why not?"
>
> **Answer:** No, 100% line coverage only means every line of code was executed at least once during the tests. It does not verify different combinations of inputs, edge cases, race conditions, logical errors, or unexpected database states.
>
> "Why do we use `task.apply(kwargs=...)` instead of `task.delay(...)` in the failure test?"
>
> **Answer:** `task.apply()` runs the task synchronously in the current process, making it easy to catch exceptions and test the task logic step-by-step. `task.delay()` sends the task to Redis asynchronously, making it run in a separate Celery worker process where exceptions are caught by Celery and not raised in the test execution context.

✓ Day Checkpoint

`pytest --cov=games --cov-report=term-missing` passes with all 3 tests green and coverage ≥ 70%. Student can explain what each test is verifying and why the mocks are necessary.